# Rider Service — Especificación de Requerimientos (SRS)

**Proyecto:** Sistema Integral de Asistencia al Motociclista · **Producto:** Rider Service — "Nunca viajes solo"
**Materia:** Aplicaciones Móviles · **Entrega:** lunes 2 de noviembre de 2026
**Fecha:** 1 de noviembre de 2026

| Capa | Tecnología |
|---|---|
| Frontend | React Native + Expo SDK 58 (Expo Router) |
| Estilos | NativeWind (Tailwind CSS para React Native) con tema centralizado |
| Lenguaje | TypeScript en modo estricto |
| Autenticación | Supabase Auth (correo y contraseña; Google OAuth con PKCE) |
| Base de datos | PostgreSQL gestionado por Supabase, con RLS en todas las tablas |
| Tiempo real | Supabase Realtime (solicitudes, bitácora, cobros y ubicación del mecánico) |
| Archivos | Supabase Storage |
| Lógica del servidor | Funciones PL/pgSQL y triggers (estados, precio, cobro y comisión) |
| Ubicación y mapas | expo-location; un único mapa Leaflet en toda la app (WebView en el celular, librería leaflet en la web) con teselas de CARTO Voyager (clave en `.env`; sin clave, Esri World Street Map). Rutas por calles y tiempo estimado con OSRM (servicio público, sin clave). |
| Contenido editorial | Archivos JSON locales (consejos y promociones) |
| Control de versiones | Git + GitHub (Pull Requests) |

## 1. Propósito y alcance

### 1.1 Propósito del producto

Los motociclistas son uno de los grupos más vulnerables ante imprevistos mecánicos en ruta. Rider Service es una plataforma bimodal que conecta en tiempo real al motociclista que necesita ayuda con el mecánico que puede prestarla. Centraliza el diagnóstico inicial, la asignación del prestador, el seguimiento del servicio y el cobro.

El valor diferencial es gestionar la incertidumbre del usuario: **quién viene, cuándo llega y cuánto cuesta**. El precio se conoce antes de pedir.

### 1.2 Modelo de negocio

Rider Service cobra una comisión del 10 % sobre cada servicio finalizado y cobrado a través de la aplicación.

- **Precio por adelantado.** Cada tipo de avería tiene una tarifa fija que el cliente ve antes de confirmar. El precio no cambia durante el servicio, como en las apps de viaje.
- **Cobro centralizado y retenido.** Al pedir, el cliente elige el medio:
  - **Mercado Pago:** el monto queda retenido al pedir. Se cobra cuando el mecánico finaliza y se devuelve si la solicitud se cancela.
  - **Efectivo:** el cliente le paga al mecánico al terminar el trabajo.
- **Registro de cada cobro:** el monto total, la comisión de la plataforma (10 %) y el neto del mecánico (90 %).

- **Liquidación al mecánico.** El neto queda registrado como saldo a liquidar. La transferencia efectiva se documenta, pero no se ejecuta.
- **Pago simulado.** El cobro no pasa por una pasarela real. El flujo completo (precio, medio de pago, retención, cobro y devolución) se registra en la base de datos, del lado del servidor, y la app identifica claramente el medio como simulado.
- **Evolución prevista: Mercado Pago real.**
  - API de Pagos con reserva del monto en la tarjeta (captura diferida): se captura al finalizar y se cancela si se anula la solicitud.
  - Credenciales y webhooks del lado del servidor (Edge Functions).
  - Después, el modelo marketplace ( `marketplace_fee` ), que divide el pago entre el mecánico y la plataforma.

### 1.3 Alcance

| Incluido | Fuera de alcance (documentado como futuro) |
|---|---|
| Autenticación real (correo y contraseña; Google para clientes) | Recuperación de contraseña por correo |
| Registro de cliente y de mecánico (con taller o de asistencia en ruta) | Notificaciones push |
| Solicitud de asistencia con GPS, precio fijo por adelantado y medio de pago | Geofencing de llegada |
| Asignación y máquina de estados con bitácora | Presupuesto con desglose y versiones |
| Seguimiento en tiempo real | Pasarela de pago real (Mercado Pago) |
| Lado del mecánico (bandeja, aceptar, llegada y finalización) | Traslado con mini-tráiler y talleres asociados |
| Perfil editable con foto | Rider Shop, comunidad y calificaciones |
| Mapa y lista de mecánicos cercanos y su ficha | Panel de administración |
| Cobro simulado retenido, comisión y devolución | |
| Mapa integrado en toda la app, ubicación del mecánico en vivo y ruta por calles (SHOULD) | |

### 1.4 Estados de implementación

| Estado | Significado |
|---|---|
| ✅ Hecho | Implementado e integrado en `main` . |
| 🟡 Parcial | Implementado con una parte pendiente, indicada en la descripción. |
| ⏳ Pendiente | Comprometido para esta entrega, todavía sin implementar. |
| — | No se implementa en esta entrega (COULD o WON'T). |

## 2. Convenciones

### 2.1 Priorización MoSCoW

| Prioridad | Significado |
|---|---|
| MUST | Sin este requisito la entrega no es válida. |
| SHOULD | De alto valor; se implementa si el tiempo lo permite. |
| COULD | Deseable, prescindible. Solo se documenta. |
| WON'T | Fuera de alcance por decisión explícita. Se documenta para evitar el scope creep. |

### 2.2 Identificadores

RF-<MÓDULO>-<NNN> (funcional), RNF-<ÁREA>-<NNN> (no funcional), RN-<NNN> (regla de negocio). Los identificadores son inmutables: un requisito conserva su ID aunque cambie su descripción o su prioridad.

## 3. Actores

| ID | Actor | Descripción | En el alcance |
|---|---|---|---|
| **ACT-01** | Cliente | Motociclista que solicita asistencia, conoce el precio antes de pedir, sigue el servicio y lo paga. | Sí |
| **ACT-02** | Mecánico | Prestador que recibe, acepta y ejecuta servicios. Puede tener **taller** (con dirección: aparece en el mapa) o hacer **asistencia en ruta** (solo declara su zona). | Sí |
| **ACT-03** | Taller asociado | Recibe motos cuando se requiere traslado. | No |
| **ACT-04** | Sistema | Validaciones, transiciones, código de solicitud, precio, cobro y comisión. | Sí (base de datos) |
| **ACT-05** | Administrador | Catálogo, moderación, disputas, liquidaciones. | No |

## 4. Mapa de pantallas

| # | Pantalla | Ruta | Requisito | Prioridad | Estado |
|---|---|---|---|---|---|
| 1 | Iniciar sesión | `/(auth)/login` | RF-VIEW-002 | MUST | ✅ |
| 2 | Registro (Usuario / Mecánico) | `/(auth)/register` | RF-VIEW-003 | MUST | ✅ |
| 3 | Inicio | `/(tabs)/inicio` | RF-VIEW-004 | MUST | ✅ |
| 4 | Perfil | `/(tabs)/perfil` | RF-VIEW-005 | MUST | 🟡 |
| 5 | Editar perfil | `/editar-perfil` | RF-VIEW-013 | MUST | ✅ |
| 6 | Solicitar asistencia (4 pasos, con precio y medio de pago) | `/solicitud/nueva` | RF-VIEW-006, RF-VIEW-014 | MUST | 🟡 |

| # | Pantalla | Ruta | Requisito | Prioridad | Estado |
|---|---|---|---|---|---|
| 7 | Solicitud confirmada | `/solicitud/confirmada` | RF-VIEW-006 | MUST | ✅ |
| 8 | Seguimiento de solicitud | `/solicitud/[id]` | RF-VIEW-008 | MUST | ✅ |
| 9 | Mis solicitudes / Mis servicios | `/(tabs)/solicitudes` | RF-VIEW-009 | MUST | ✅ |
| 10 | Mecánicos cerca tuyo (Mapa \| Lista) | `/(tabs)/mapa` | RF-VIEW-007 | SHOULD | ✅ |
| 11 | Ficha del mecánico (modal) | desde `/(tabs)/mapa` | RF-VIEW-010 | SHOULD | 🟡 |
| 12 | Mis ganancias (mecánico) | `/ganancias` | RF-VIEW-015 | SHOULD | ⏳ |
| — | Selección de rol | `/(auth)/role` | RF-VIEW-001 | WON'T | — |

No hay una pantalla de pago aparte: el medio de pago se elige en el paso 4 de la solicitud y el cobro se completa solo al finalizar (RF-VIEW-014).

### 4.1 Definición de "pantalla terminada"

Una pantalla está terminada cuando:

- usa exclusivamente el tema de NativeWind (colores, medidas y tipografías), sin valores escritos a mano;
- obtiene sus datos solo a través de `/services`;
- contempla los estados de carga, de lista vacía y de error;
- es navegable, y el botón Atrás de Android se comporta de forma coherente;
- no se rompe con textos largos ni campos nulos;
- no genera errores ni advertencias en consola, y pasa tsc y expo lint;
- fue probada en Android, en un dispositivo físico o un emulador.

## 5. Requisitos funcionales

### 5.1 Pantallas

| ID | Requisito | Prioridad | Estado | Notas |
|---|---|---|---|---|
| **RF-VIEW-001** | Pantalla de selección de rol | WON'T | — | No se implementa: el rol se elige con el conmutador Usuario/Mecánico del registro y queda en la cuenta. |
| **RF-VIEW-002** | Inicio de sesión | MUST | ✅ | Correo y contraseña reales, y botón de Google funcionando (RF-AUTH-007). |
| **RF-VIEW-003** | Registro con variantes Usuario y Mecánico | MUST | ✅ | La variante Mecánico pregunta "¿Cómo trabajás?":<br>• **Tengo taller:** nombre del taller y ubicación marcada en un mapa, con GPS o pin. La dirección se completa sola y se puede editar.<br>• **Asistencia en ruta:** sin dirección.<br>Las dos piden la zona de trabajo, las especialidades y una descripción. El botón de Google aparece solo en la variante Usuario. |
| **RF-VIEW-004** | Inicio | MUST | ✅ | Saludo con el perfil real; mecánicos destacados desde la base; consejos y promociones desde JSON. Tiene 3 accesos rápidos:<br>• "Mecánicos cerca tuyo" (abre la vista Lista);<br>• "Ver mapa" (abre la vista Mapa);<br>• "Mis solicitudes". |

| ID | Requisito | Prioridad | Estado | Notas |
|---|---|---|---|---|
| **RF-VIEW-005** | Perfil | MUST | 🟡 | Datos y foto reales; confirmación al cerrar sesión. Pendiente: las estadísticas se muestran en cero. |
| **RF-VIEW-006** | Flujo de solicitud en 4 pasos | MUST | 🟡 | Funcional, con GPS. El paso 1 muestra el precio de cada tipo de avería. El paso 4 muestra el precio total y el medio de pago. Pendiente: el botón Atrás de Android debe volver al paso anterior. |
| **RF-VIEW-007** | Mecánicos cerca tuyo | SHOULD | ✅ | Una pantalla con dos vistas:<br>• **Mapa:** los talleres con ubicación.<br>• **Lista:** todos los mecánicos. Los de asistencia en ruta llevan la etiqueta "Asistencia en ruta · zona".<br>Distancia desde el GPS, filtro por especialidad y ficha del mecánico. Solo usa datos de la base. |
| **RF-VIEW-008** | Seguimiento de solicitud | MUST | ✅ | Estado, línea de tiempo de 6 estados, mecánico, contacto, precio y estado del cobro en tiempo real, y cancelación. Incluye mapa integrado con el cliente, el mecánico en vivo y la ruta (RF-GPS-005). |
| **RF-VIEW-009** | Mis solicitudes / Mis servicios | MUST | ✅ | Cliente: activas y finalizadas. Mecánico: "Mis servicios" y "Disponibles". Cada solicitud muestra su precio y el estado del cobro. |
| **RF-VIEW-010** | Detalle de mecánico | SHOULD | 🟡 | Ficha en un modal: datos, dirección o zona, distancia, descripción, especialidades y llamada. Pendiente: WhatsApp y abrirla desde los destacados del Inicio. |
| **RF-VIEW-011** | Estructura de navegación con botón SOS | MUST | ✅ | Las 4 pestañas y el SOS central activos. |
| **RF-VIEW-012** | Contenido desde la capa de servicios | MUST | ✅ | Datos operativos desde Supabase; contenido editorial desde JSON. Ninguna vista accede a Supabase ni a un JSON directamente. |
| **RF-VIEW-013** | Editar perfil | MUST | ✅ | Nombre y teléfono con validación. |
| **RF-VIEW-014** | Pago del servicio | MUST | ✅ | Integrado en el paso 4 de la solicitud: precio fijo y medio de pago (Mercado Pago simulado o efectivo). Ver 5.9. |
| **RF-VIEW-015** | Mis ganancias del mecánico | SHOULD | ⏳ | Ver RF-PAY-004. |

### 5.2 Autenticación y sesión

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-AUTH-001** | Rol con efecto real | MUST | ✅ | El rol de la cuenta determina las pantallas y acciones disponibles. |
| **RF-AUTH-002** | Inicio de sesión | MUST | ✅ | Contra Supabase Auth. El mensaje de error no revela qué campo falló. La sesión se guarda en el dispositivo. |
| **RF-AUTH-003** | Registro de cliente | MUST | ✅ | Validación de campos, mínimo de 6 caracteres, coincidencia de contraseñas y términos. |

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-AUTH-004** | Registro de mecánico | MUST | ✅ | Un trigger crea el perfil y la ficha del mecánico con zona, especialidades y descripción. Si tiene taller, guarda también la dirección y las coordenadas. |
| **RF-AUTH-005** | Cierre de sesión | MUST | ✅ | Pide confirmación en un modal; al confirmar limpia la sesión y vuelve al login. |
| **RF-AUTH-006** | Control de acceso por rol | MUST | ✅ | Reforzado con RLS en la base. |
| **RF-AUTH-007** | Inicio de sesión con Google | MUST | ✅ | Ver detalle abajo. |
| **RF-AUTH-008** | Recuperación de contraseña | COULD | — | Requiere configurar el correo de Supabase con enlaces de vuelta a la app. |
| **RF-AUTH-009** | Validación de formularios | MUST | ✅ | Mensajes de error en pantalla. |

**RF-AUTH-007: inicio de sesión con Google.** OAuth 2.0 mediante Supabase Auth, con el proveedor Google y el navegador del sistema (expo-web-browser, con flujo PKCE).

Criterios de aceptación (todos verificados en Android con Expo Go):

- El botón "Continuar con Google" del login abre la selección de cuenta de Google y, al volver, ingresa al Inicio con la sesión creada.
- La cuenta creada con Google tiene siempre el rol cliente (RN-015). El nombre y la foto se toman de Google.
- El registro de mecánicos se hace solo con el formulario completo.
- Si el correo ya existía con contraseña, Supabase vincula las dos identidades a la misma cuenta.
- El client secret de Google se configura en Supabase y nunca forma parte del repositorio.
- En Supabase, la Site URL es `riderservice://` y las Redirect URLs habilitadas son `exp://**`, `riderservice://**` y `http://localhost:8081/**`.
- En Expo Go la app se levanta con `npx expo start --tunnel` (ngrok), porque Supabase no acepta direcciones de vuelta con IP.
- Si el usuario cancela, vuelve al login sin error.

### 5.3 Perfil

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-PROF-001** | Perfil con datos propios | MUST | ✅ | RLS impide leer perfiles ajenos. |
| **RF-PROF-002** | Edición de datos personales | MUST | ✅ | Nombre y teléfono. La localidad queda como COULD. |
| **RF-PROF-003** | Disponibilidad del mecánico | SHOULD | ⏳ | El mecánico activa o desactiva su disponibilidad. Un mecánico no disponible no ve solicitudes nuevas (RN-003). |
| **RF-PROF-004** | Mis motos | WON'T | — | Futuro. |
| **RF-PROF-005** | Mecánicos favoritos | WON'T | — | Futuro. |

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-PROF-006** | Notificaciones y seguridad | WON'T | — | Futuro. |
| **RF-PROF-007** | Foto de perfil | MUST | ✅ | Hasta 5 MB, en formato JPG, PNG o WebP. Se guarda en el bucket `avatars`, en la carpeta del usuario. |

### 5.4 Solicitud de asistencia

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-ASSIST-001** | Flujo con validación por paso | MUST | ✅ | No se avanza sin completar los campos obligatorios. |
| **RF-ASSIST-002** | Tipo de avería obligatorio | MUST | ✅ | 8 categorías desde la base, cada una con su tarifa fija. |
| **RF-ASSIST-003** | Ubicación real por GPS | MUST | ✅ | Con expo-location. Si el GPS falla, se informa al usuario. |
| **RF-ASSIST-004** | Descripción opcional | MUST | ✅ | Hasta 200 caracteres. |
| **RF-ASSIST-005** | Creación de la solicitud | MUST | ✅ | Código RS-AAAA-NNNN, fecha y hora, estado inicial pendiente. La base fija el precio con la tarifa y registra el cobro: retenido (Mercado Pago) o pendiente (efectivo). |
| **RF-ASSIST-006** | Selección de vehículo | WON'T | — | Depende de RF-PROF-004. |
| **RF-ASSIST-007** | Evidencia fotográfica | COULD | — | El bucket privado archivos ya existe. |
| **RF-ASSIST-008** | Cancelar solicitud | MUST | ✅ | Antes de trabajando (RN-009), con motivo, y queda registrada en la bitácora. Lo retenido se devuelve. |
| **RF-SOS-001** | Acceso rápido SOS | MUST | ✅ | El botón central abre el flujo de solicitud. |

### 5.5 Asignación

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-MATCH-001** | Mecánicos disponibles ven las solicitudes pendientes | MUST | ✅ | Bandeja "Disponibles", restringida por RLS. Les llega a todos los mecánicos disponibles, tengan taller o no. |
| **RF-MATCH-002** | Notificación push de oportunidades | WON'T | — | Futuro. La bandeja se actualiza por Realtime. |
| **RF-MATCH-003** | Aceptar servicio | MUST | ✅ | Antes de aceptar, el mecánico ve el precio, su neto (90 %) y el medio de pago. Al aceptar informa el tiempo estimado de llegada (ETA) y sale en camino. |
| **RF-MATCH-004** | Evitar asignación múltiple | MUST | ✅ | Actualización condicional atómica en aceptar_solicitud (RN-002). |

### 5.6 Máquina de estados

```
pendiente → aceptado → en_camino → llegue → trabajando → finalizado
   └─────────────┴──────────────┴──────────┘
                          ↓
                      cancelada
```

La bitácora registra los 6 estados. Para el mecánico, la base agrupa pasos y quedan **3 acciones**:

- **"Aceptar y salir en camino"** registra aceptado y en camino.
- **"Llegué, empezar el trabajo"** registra la llegada y trabajando.
- **"Finalizar trabajo"**.

| ID | Requisito | Prioridad | Estado |
|---|---|---|---|
| **RF-STATE-001** | Gestión de estados con transiciones validadas en la base | MUST | ✅ |
| **RF-STATE-002** | → En camino (solo el mecánico asignado). Se registra automáticamente al aceptar. | MUST | ✅ |
| **RF-STATE-003** | → Llegué | MUST | ✅ |
| **RF-STATE-004** | → Trabajando. Se registra automáticamente al registrar la llegada. | MUST | ✅ |
| **RF-STATE-005** | → Finalizado. Completa el cobro (RF-PAY-002). | MUST | ✅ |

### 5.7 Seguimiento

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-TRACK-001** | Seguimiento con datos reales | MUST | ✅ | |
| **RF-TRACK-002** | Línea de tiempo en tiempo real | MUST | ✅ | Supabase Realtime. |
| **RF-TRACK-003** | Contacto con el mecánico | MUST | ✅ | Llamada mediante Linking. |
| **RF-TRACK-004** | Historial de solicitudes | MUST | ✅ | |

### 5.8 GPS y mapas

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-GPS-001** | Ver la ubicación del mecánico en el mapa | SHOULD | ✅ | El cliente ve en el seguimiento al mecánico moverse en vivo, con la ruta por las calles hasta su ubicación y el tiempo estimado ("Llega en ~X min · Y km"). Si el servicio de rutas no responde, se muestra una línea recta. |
| **RF-GPS-002** | Emitir la ubicación del mecánico en vivo | SHOULD | ✅ | Mientras el servicio está en curso y el mecánico tiene la app abierta, su posición se guarda cada 5 s o 15 m en `ubicaciones_mecanico` y se publica por Realtime. Se borra al finalizar o cancelar. |
| **RF-GPS-003** | Geofencing de llegada | WON'T | — | Futuro. |
| **RF-GPS-004** | Ubicación del cliente para el mecánico | SHOULD | ✅ | El mecánico ve en un mapa dónde está el cliente desde que la solicitud aparece en "Disponibles" y durante el viaje. |

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-GPS-005** | Mapa único integrado | SHOULD | ✅ | Un solo mapa Leaflet en toda la app: selector de ubicación, registro del taller, seguimiento y mapa de mecánicos, igual en el celular y en la web. No requiere clave de Google y funciona en Expo Go. Los mapas se ven dentro de la app, sin enlaces externos. |

### 5.9 Precio, cobro y comisión

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-QUOTE-001** | Precio del servicio por adelantado | MUST | ✅ | Cada tipo de avería tiene una tarifa fija ( `categorias_averia.tarifa` ). Al crear la solicitud la base la copia a `solicitudes.monto_final` ; la app no puede fijarla ni cambiarla. |
| **RF-QUOTE-002** | Consultar el precio | MUST | ✅ | El cliente lo ve en los pasos 1 y 4 de la solicitud, en el seguimiento y en Mis solicitudes. El mecánico lo ve antes de aceptar. |
| **RF-QUOTE-003** | Presupuesto con desglose y aceptación previa | WON'T | — | Desglose en mano de obra, repuestos y tasa; con versiones. Futuro. |
| **RF-PAY-001** | Cobro simulado al pedir | MUST | ✅ | Ver detalle abajo. |
| **RF-PAY-002** | Registro del cobro con comisión | MUST | ✅ | Ver detalle abajo. |
| **RF-PAY-003** | Estado del cobro visible | MUST | ✅ | Seguimiento y Mis solicitudes muestran "Pago retenido", "Efectivo al finalizar", "Pagado", "Devuelto" o "Sin cargo", en tiempo real. |
| **RF-PAY-004** | Ganancias del mecánico | SHOULD | ⏳ | El mecánico ve sus servicios cobrados, el neto de cada uno (90 %) y el total a liquidar. Hoy ve el neto de cada servicio en su seguimiento. |
| **RF-PAY-005** | Integración con Mercado Pago | WON'T | — | Futuro: API de Pagos con reserva del monto (captura al finalizar o cancelación), credenciales y webhooks en Edge Functions. |
| **RF-PAY-006** | Pago dividido marketplace | WON'T | — | `marketplace_fee` con la cuenta del mecánico vinculada por OAuth. Futuro. |

**RF-PAY-001: cobro simulado al pedir.** En el paso 4 el cliente ve el precio y elige el medio:

- **Mercado Pago (simulado):** el botón dice "Confirmar y pagar $X" y el monto queda retenido. Se identifica claramente como pago de prueba y no pide datos de tarjeta.
- **Efectivo:** queda pendiente y se paga al mecánico al terminar.

**RF-PAY-002: registro del cobro con comisión.** Lo hace la base con triggers, nunca la app:

- **Al crear la solicitud:** registra el cobro con la comisión (10 %) y el neto del mecánico (90 %), en estado retenido o pendiente según el medio.
- **Al aceptar:** asocia el cobro al mecánico.
- **Al finalizar:** lo marca cobrado.
- **Al cancelar:** lo marca devuelto (Mercado Pago) o anulado (efectivo).
- **Siempre:** un cobro por solicitud y una sola vez (RN-013).

### 5.10 Otros módulos

| ID | Requisito | Prioridad |
|---|---|---|
| **RF-RATING-001/002** | Calificación del servicio y promedio del mecánico | COULD |
| **RF-NOTIF-001** | Avisos dentro de la app (campana) | COULD |
| **RF-NOTIF-002** | Notificaciones push | WON'T |
| **RF-TRANSFER-001 a 003** | Traslado con mini-tráiler | WON'T |
| **RF-SHOP-001 a 005** | Rider Shop | WON'T |
| **RF-COMM-001** | Comunidad | WON'T |

### 5.11 Datos, archivos y seguridad

| ID | Requisito | Prioridad | Estado | Descripción |
|---|---|---|---|---|
| **RF-DATA-001** | Persistencia real | MUST | ✅ | PostgreSQL. |
| **RF-DATA-002** | Estado de sesión compartido | MUST | ✅ | useSesion y RequiereSesion. |
| **RF-DATA-003** | Sesión persistente en el dispositivo | MUST | ✅ | AsyncStorage. |
| **RF-DATA-004** | Bitácora de estados | MUST | ✅ | historial_solicitudes: estado anterior, estado nuevo, fecha y hora, actor y nota. |
| **RF-FILE-001** | Archivos en Storage | MUST | ✅ | avatars es público con escritura por carpeta del usuario; archivos es privado. |
| **RF-SEC-001** | Acceso a datos mediante RLS | MUST | ✅ | Ver detalle abajo. |

**RF-SEC-001.** RLS controla el acceso a los datos:

- un cliente no ve ni modifica servicios ajenos;
- un mecánico solo opera los servicios que tiene asignados;
- los cobros solo los ven el cliente y el mecánico del servicio. El mecánico los ve desde que acepta.
- la app no puede insertar ni modificar cobros: solo los escriben los triggers de la base.

## 6. Requisitos no funcionales

| ID | Requisito | Prioridad | Estado | Criterio de aceptación |
|---|---|---|---|---|
| **RNF-UX-001** | Sistema de diseño consistente | MUST | 🟡 | Tema en tailwind.config.js; los colores para íconos salen de src/lib/tema.ts. Pendiente: quitar los hex que quedan en los componentes (59). |
| **RNF-UX-002** | Fidelidad a la maqueta | MUST | ✅ | |

| ID | Requisito | Prioridad | Estado | Criterio de aceptación |
|---|---|---|---|---|
| **RNF-UX-003** | Estados de carga, vacío y error | MUST | ✅ | Ninguna pantalla queda en blanco. |
| **RNF-UX-004** | Respuesta táctil de los controles | MUST | ✅ | |
| **RNF-UX-005** | Legibilidad en exteriores | SHOULD | ✅ | Contraste de 4.5:1 o más y área táctil de 44×44 pt o más. |
| **RNF-UX-006** | Teclado que no tapa el campo activo | SHOULD | ✅ | |
| **RNF-UX-007** | Accesibilidad | COULD | 🟡 | Etiquetas, roles y descripciones de accesibilidad en botones, chips, accesos rápidos, filtros, mapa y ficha. |
| **RNF-UX-008** | Flujo de emergencia | SHOULD | ✅ | 60 segundos o menos y 8 interacciones o menos. |
| **RNF-PERF-001** | Fluidez de la interfaz | MUST | ✅ | Navegación en 300 ms o menos. |
| **RNF-PERF-002** | Arranque | SHOULD | ✅ | Menos de 3 segundos. |
| **RNF-COMP-001** | Android | MUST | ✅ | Android 10 o superior, Expo Go (SDK 58) y dispositivo físico. |
| **RNF-COMP-002** | iOS | SHOULD | 🟡 | Código compatible; no se verificó en un dispositivo. |
| **RNF-COMP-003** | Diseño adaptable | MUST | ✅ | Pantallas de 5" a 6.7", respetando las áreas seguras. |
| **RNF-SEC-001** | Protección de secretos | MUST | ✅ | Ver detalle abajo. |
| **RNF-SEC-002** | Autenticación segura | MUST | ✅ | Supabase Auth; la contraseña nunca se guarda en el dispositivo. |
| **RNF-SEC-003** | Autorización por políticas | MUST | ✅ | RLS en todas las tablas con políticas explícitas. |
| **RNF-SEC-004** | Seguridad de archivos | MUST | ✅ | Escritura solo en la carpeta propia. |
| **RNF-SEC-005** | Montos calculados en el servidor | MUST | ✅ | El precio, la comisión, el neto y el estado del cobro los fija la base; la app no puede enviarlos ni modificarlos. |
| **RNF-MAIN-001** | Control de versiones | MUST | ✅ | main siempre ejecutable. |
| **RNF-MAIN-002** | Integración por Pull Request con revisión | MUST | ✅ | |
| **RNF-MAIN-003** | Modularidad | MUST | ✅ | /app, /components, /services, /types, /data, /lib, /hooks. Solo /services accede a datos. |
| **RNF-MAIN-004** | TypeScript en modo estricto | SHOULD | ✅ | |
| **RNF-MAIN-005** | ESLint | SHOULD | ✅ | expo lint sin errores ni advertencias. |
| **RNF-MAIN-006** | README completo | MUST | 🟡 | Pendiente: integrantes, alcance y cómo levantar la app con túnel para Google. |

| ID | Requisito | Prioridad | Estado | Criterio de aceptación |
|---|---|---|---|---|
| **RNF-MAIN-007** | Migraciones versionadas | MUST | ✅ | Todo cambio de esquema queda en supabase/migrations/. |
| **RNF-SCAL-001** | Independencia del origen de datos | MUST | ✅ | Cambiar el origen de los datos solo afecta a /services. |
| **RNF-SCAL-002** | Categorías y especialidades declarativas | SHOULD | ✅ | Las tarifas también son datos de la tabla de categorías. |
| **RNF-REL-001** | Robustez ante datos | MUST | ✅ | Nulos, listas vacías y textos largos. |
| **RNF-REL-002** | Manejo de errores de operación | MUST | ✅ | Una operación fallida nunca se informa como exitosa. |
| **RNF-REL-003** | Persistencia confiable | MUST | ✅ | |
| **RNF-TRACE-001 a 003** | Trazabilidad | MUST | ✅ | Los commits y PR referencian el ID del requisito. |
| **RNF-TEST-002** | Datos de prueba | MUST | ✅ | Seed versionado con 10 talleres de Pilar y Tortuguitas (datos públicos de Google Maps), con especialidades, teléfono, horarios y uno no disponible. |
| **RNF-TEST-003** | Estados determinísticos | MUST | ✅ | scripts/verificar-supabase.mjs. |
| **RNF-PRIV-001** | Minimización de datos | MUST | ✅ | |
| **RNF-PRIV-002** | Protección de la ubicación | MUST | ✅ | El permiso se pide al momento de usarla, explicando el motivo. |
| **RNF-AI-001** | Transparencia sobre el uso de IA | SHOULD | ⏳ | Documentar en el README. |
| **RNF-AI-002** | Validación humana | MUST | ✅ | |
| **RNF-AI-003** | Protección de información | MUST | ✅ | |

**RNF-SEC-001.** Ningún secreto forma parte del repositorio:

- `.env` está en `.gitignore`;
- la clave publishable de Supabase es pública por diseño y figura en `.env.example`;
- las claves privadas (`service_role` y el client secret de Google) nunca entran al repositorio.

## 7. Reglas de negocio

| ID | Regla | Prioridad | Estado |
|---|---|---|---|
| **RN-001** | Toda solicitud está asociada a un cliente autenticado. | MUST | ✅ |
| **RN-002** | Una solicitud no puede tener dos mecánicos asignados. | MUST | ✅ |
| **RN-003** | Un mecánico no disponible no recibe solicitudes nuevas. | MUST | ✅ |

| ID | Regla | Prioridad | Estado |
|---|---|---|---|
| **RN-004** | Los estados respetan el flujo definido. | MUST | ✅ |
| **RN-005** | Finalizado es terminal. | MUST | ✅ |
| **RN-006** | Una venta no supera el stock disponible. | WON'T | — |
| **RN-007** | Un servicio se considera pagado solo cuando la base registra su cobro como cobrado. | MUST | ✅ |
| **RN-008** | Los datos de un servicio son visibles solo para sus actores. | MUST | ✅ |
| **RN-009** | Solo se cancela antes de trabajando. Como la llegada inicia el trabajo, en la práctica se puede cancelar hasta que llega el mecánico. | MUST | ✅ |
| **RN-010** | Un cliente tiene como máximo una solicitud activa. | MUST | ✅ |
| **RN-011** | Solo el cliente califica, y solo un servicio finalizado. | COULD | — |
| **RN-012** | La comisión es el 10 % del precio del servicio; el neto del mecánico es el 90 %. El neto se redondea a 2 decimales y la comisión absorbe la diferencia. | MUST | ✅ |
| **RN-013** | Cada solicitud tiene un único cobro. Se retiene (o queda pendiente) al pedir, se cobra solo al finalizar y, si se cancela, se devuelve (o se anula). | MUST | ✅ |
| **RN-014** | El precio lo fija la base con la tarifa de la categoría al crear la solicitud; ni el cliente ni el mecánico pueden modificarlo. | MUST | ✅ |
| **RN-015** | Una cuenta creada con Google tiene el rol cliente; nunca se le crea ficha de mecánico. | MUST | ✅ |
| **RN-016** | Un mecánico con taller tiene dirección y coordenadas y aparece en el mapa. Uno de asistencia en ruta solo declara su zona: aparece en la lista, pero no en el mapa. Los dos reciben solicitudes. | MUST | ✅ |

## 8. Modelo de datos

| Tabla | Contenido relevante |
|---|---|
| `perfiles` | Nombre, teléfono y foto. Con Google, el nombre y la foto salen de la cuenta de Google. |
| `mecanicos` | Nombre del taller, descripción, teléfono, localidad (zona de trabajo), **direccion**, **latitud** y **longitud** (solo si tiene taller), especialidades y disponibilidad. |
| `categorias_averia` | Nombre, descripción, ícono y **tarifa** (precio fijo en pesos). |
| `solicitudes` | Cliente, mecánico, categoría, ubicación, estado, ETA, motivo de cancelación, **monto_final** (precio fijado al pedir) y **medio_pago** (mercado_pago o efectivo). |
| `historial_solicitudes` | Bitácora de estados: anterior, nuevo, fecha, actor y nota. |
| `ubicaciones_mecanico` | Última posición del mecánico por solicitud (latitud, longitud y hora). Solo la escribe el mecánico asignado mediante `actualizar_ubicacion_mecanico`; la leen el cliente y el mecánico del servicio. Se borra al finalizar o cancelar. |
| `pagos` | Uno por solicitud: cliente, mecánico, monto, comisión, neto del mecánico, medio y estado (retenido, pendiente, cobrado, devuelto o anulado). Con RLS de solo lectura para el cliente y el mecánico del servicio. |

**Funciones y triggers:**

- `crear_perfil_usuario` : crea el perfil y, si es mecánico registrado con el formulario, su ficha con zona, dirección y coordenadas. Con Google toma el nombre y la foto y nunca crea un mecánico (RN-015).
- `fn_tarifa_solicitud` (antes de crear la solicitud): fija `monto_final` con la tarifa de la categoría (RN-014).
- `fn_crear_pago_solicitud` (después de crearla): registra el cobro retenido o pendiente con su comisión (RN-012, RN-013).
- `fn_sincronizar_pago` (al cambiar el estado o el mecánico): asocia el cobro al mecánico y lo pasa a cobrado, devuelto o anulado.
- `cambiar_estado_solicitud` : valida quién opera, registra la bitácora y agrupa pasos (aceptar → en camino; llegué → trabajando).
- `actualizar_ubicacion_mecanico` : guarda la posición del mecánico asignado, solo con el servicio en curso (RF-GPS-002).
- `fn_limpiar_ubicacion_mecanico` (al finalizar o cancelar): borra la ubicación guardada.

**Migraciones principales:**

- `20261031170000_mecanicos_direccion_taller`
- `20261031180000_seed_talleres_pilar_tortuguitas`
- `20261031200000_google_oauth_perfil`
- `20261101000000_monto_final_y_pagos`
- `20261101010000_tarifa_y_cobro_retenido`
- `20261101020000_ubicacion_mecanico_en_vivo`

## 9. Recorridos de verificación

| # | Recorrido | Prioridad |
|---|---|---|
| 1 | Login con correo → Inicio → cerrar sesión (con confirmación) → Login | MUST |
| 2 | Login → Continuar con Google → Inicio con nombre y foto de Google (Expo levantado con túnel) | MUST |
| 3 | Registro de cliente y de mecánico (con taller o de ruta) → Login con la cuenta nueva | MUST |
| 4 | Cliente: SOS → 4 pasos con GPS. Ve el precio y elige el medio de pago → Confirmada → Seguimiento en pendiente con "Pago retenido" | MUST |
| 5 | Mecánico: Disponibles (ve el precio y su neto) → Aceptar con ETA (queda en camino) → Llegué (queda trabajando) → Finalizar. El cliente ve cada cambio en tiempo real. | MUST |
| 6 | Cliente: el servicio finaliza → el cobro pasa a "Pagado" (con Mercado Pago simulado o en efectivo) | MUST |
| 7 | Mecánico: Mis ganancias muestra el neto del servicio cobrado (90 %) | SHOULD |
| 8 | Cliente: cancela una solicitud antes de que llegue el mecánico → queda cancelada con el motivo y el cobro "Devuelto" | MUST |
| 9 | Perfil → Editar perfil → guardar → Perfil actualizado; cambiar la foto | MUST |
| 10 | Inicio → Mecánicos cerca tuyo (vista Lista) → ficha del mecánico → volver | SHOULD |
| 11 | Registro de mecánico con taller → el taller aparece en la vista Mapa | SHOULD |

| # | Recorrido | Prioridad |
|---|---|---|
| 12 | Mecánico acepta → el cliente ve al mecánico moverse en el mapa del seguimiento, con la ruta por calles y el tiempo estimado | SHOULD |

## 10. Criterio de aceptación global

La entrega es conforme cuando:

1. Todos los requisitos MUST están en ✅.
2. Los recorridos MUST se completan sin errores en Android.
3. `npx tsc --noEmit` y `npx expo lint` terminan sin errores.
4. No hay colores ni medidas escritos a mano en los componentes.
5. Ningún secreto privado está versionado.
6. El README declara el alcance, la instalación, la estructura y los integrantes.
7. Todo cambio entró a `main` mediante un Pull Request revisado.